Guion de la presentación

Generative AI-Assisted Malware Disassembly and Remediation

Máster en Ciberseguridad — Análisis de Malware · 3 autores · 20 min

Diapositivas en inglés · guion en español.
Cada apartado = una diapositiva, con los dos bloques (qué decir / profundizar).

Índice de diapositivas

Cómo usar este guion . . . 3
Slide 1 — Portada (Title) . . . 3
Slide 2 — Agenda . . . 3
Slide 3 — Divisor: Context & Motivation . . . 3
Slide 4 — Reverse engineering is slow, scarce and expensive . . . 4
Slide 5 — The promise of LLMs . . . 4
Slide 6 — Divisor: The Toolchain . . . 4
Slide 7 — The classic RE stack . . . 4
Slide 8 — The LLM-augmented stack (diagrama) . . . 5
Slide 9 — Divisor: Manual vs. AI-Augmented Workflow . . . 5
Slide 10 — Two lanes, same destination . . . 5
Slide 11 — Comparison table . . . 6
Slide 12 — Divisor: Evidence — Speed, Accuracy & Hallucinations . . . 6
Slide 13 — Speed & comprehension . . . 6
Slide 14 — Remediation & patching . . . 6
Slide 15 — The hallucination problem . . . 7
Slide 16 — Divisor: Live Demo . . . 7
Slide 17 — Demo: before / after . . . 7
Slide 18 — Play recorded demo . . . 8
Slide 19 — Divisor: Takeaways . . . 8
Slide 20 — Takeaways . . . 8
Slide 21 — Credits & contributions . . . 8
Slide 22 — Thank you / Q&A . . . 9
Bibliografía y dónde ampliar información . . . 10

Cómo usar este guion

Lee primero el bloque 🎤 **Qué decir** para cada diapositiva: es el discurso base (3–6 frases), pensado para traducir fácil al inglés al hablar (las slides ya están en inglés → punto extra del profesor). El bloque 📚 **Profundización** es para que **entiendas** de verdad el tema y puedas responder preguntas: trae las cifras exactas, sus matices y las referencias. No hace falta decir todo lo de profundización en voz alta.

Reparto sugerido (3 ponentes): Autor 1 → slides 1–8 · Autor 2 → slides 9–15 · Autor 3 → slides 16–18. **Ritmo:** 1 min por slide de contenido + 4 min de vídeo-demo. Los divisores de agenda son transiciones de 5–10 s («ya hemos visto X, ahora vamos a Y»).

Slide 1 — Portada (Title)

🎤 QUÉ DECIR

«Buenos días. Somos [nombres] y nuestra presentación trata sobre **IA generativa aplicada al desensamblado y la remediación de malware**: cómo los grandes modelos de lenguaje ayudan a hacer ingeniería inversa y a proponer parches más rápido, y también dónde siguen fallando. El subtítulo lo resume: aceleran el trabajo, pero no sustituyen al analista.»

📚 PROFUNDIZACIÓN Y REFERENCIAS

Contexto del tema (enunciado): los LLM ayudan al analista a **interpretar código decompilado** (p. ej. el pseudo-C de Ghidra), **identificar la vulnerabilidad explotada** y **proponer mitigaciones/parches**. Vamos a contrastar el flujo manual con el aumentado por IA evaluando **velocidad, precisión y riesgo de alucinaciones**, y cerramos con una demo grabada. Mantén el marco honesto desde el inicio: «co-piloto», no «piloto automático».

Slide 2 — Agenda

🎤 QUÉ DECIR

«Esta es la hoja de ruta: primero el **contexto** y por qué esto importa; luego las **herramientas** que se usan; después comparamos el **flujo manual con el aumentado por IA**; repasamos la **evidencia** (velocidad, precisión y alucinaciones); hacemos una **demo**; y cerramos con **conclusiones**.»

📚 PROFUNDIZACIÓN Y REFERENCIAS

La agenda se repite en cada cambio de sección resaltando dónde estamos (requisito de organización del profesor → puntúa en «Presentación y Organización»). Son 6 secciones. Di la transición en una frase y pasa rápido.

Slide 3 — Divisor: Context & Motivation

🎤 QUÉ DECIR

«Empecemos situando el problema: ¿por qué necesitamos acelerar esto?»

📚 PROFUNDIZACIÓN Y REFERENCIAS

Transición. No te detengas.

Slide 4 — Reverse engineering is slow, scarce and expensive

🎤 **QUÉ DECIR**

«La ingeniería inversa de malware es un cuello de botella: desensamblar, decompilar y analizar dinámicamente una muestra puede llevar **horas o días**, y depende de expertos escasos. Al mismo tiempo aparecen **cientos de miles** de muestras nuevas al día y hay un **backlog enorme de CVEs sin parchear**. Leer ensamblador o pseudocódigo ruidoso es difícil y propenso a errores. La pregunta es: ¿puede la IA generativa reducir el tiempo hasta entender y hasta parchear, sin mentirnos?»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

- Volumen: AV-TEST registra del orden de **450.000+ nuevas muestras de malware/PUA al día** (ver AV-TEST malware statistics). Usa «cientos de miles/día» para ir sobre seguro.
- «Sin mentirnos» anticipa el hilo conductor: las alucinaciones (slide 15).
- **Decompilación vs. desensamblado**: desensamblar = pasar bytes a instrucciones de ensamblador; decompilar = reconstruir pseudo-C de más alto nivel. El decompilador **adivina** tipos y flujo → salida aproximada, sin nombres ni comentarios originales. Esa es la entrada ruidosa que damos al LLM.

Slide 5 — The promise of LLMs

🎤 **QUÉ DECIR**

«¿Qué aporta un LLM aquí? Tres cosas: **explicar** el pseudocódigo en lenguaje natural, **renombrar** funciones y variables para recuperar la intención del código, y **proponer** una hipótesis del fallo y un borrador de mitigación o parche. Pero —y esto es clave en toda la charla— el LLM es un **acelerador y co-piloto**, no un sustituto del analista: todo lo que diga hay que **verificarlo contra el binario**.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Este es el **framing honesto** que pide un trabajo de calidad y que nos cubre ante preguntas. El valor del LLM es sobre todo de **comprensión y triaje**, no de garantía de corrección. Enlaza con la evidencia posterior: gran mejora para no-expertos, mejora pequeña para expertos.

Slide 6 — Divisor: The Toolchain

🎤 **QUÉ DECIR**

«Para entendernos, situemos rápido las herramientas que se usan.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Transición de 5 s.

Slide 7 — The classic RE stack

🎤 **QUÉ DECIR**

«El punto de partida es el stack clásico. En **estático**: decompiladores como **Ghidra** —gratuito, de la NSA—, IDA Pro o Binary Ninja, más utilidades como `strings` y la lectura de cabeceras PE/ELF e imports. En **dinámico**: depuradores como x64dbg o GDB, y sandboxes como Cuckoo/CAPE en máquinas virtuales aisladas. Una idea importante: el **pseudocódigo no es el código fuente**; el decompilador adivina tipos y flujo. Ese es precisamente el texto ruidoso que luego le pedimos interpretar a la IA.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

- **Ghidra**: suite de RE de la NSA, open-source (ghidra-sre.org); su decompilador genera pseudo-C. Es el estándar gratuito y el que usa la mayoría de puentes con LLM.
- **Sandbox**: ejecución controlada para observar comportamiento (red, ficheros, registro) sin leer código. Complementa el estático.
- Recalca el punto del pseudocódigo porque justifica por qué el LLM a veces alucina: trabaja sobre una reconstrucción imperfecta.

Slide 8 — The LLM-augmented stack (diagrama)

🎤 **QUÉ DECIR**

«La clave es que el LLM no se usa copiando y pegando: se **conecta** a la herramienta. El diagrama muestra el flujo: del **binario** a **Ghidra**, y a través de un **puente MCP** el modelo accede a la API de Ghidra. El ejemplo más conocido es **GhidraMCP**, de LaurieWired, que expone funciones como listar y decompilar funciones, renombrar o mirar referencias cruzadas. Así el agente **itera** —decompila, explica, renombra, vuelve a leer, propone parche— siempre con un **humano en el bucle** que acepta o rechaza cada cambio.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

- **MCP (Model Context Protocol)**: protocolo estándar para dar herramientas/datos a un LLM. **GhidraMCP** (github.com/LaurieWired/GhidraMCP) = plugin de Ghidra + servidor puente; permite a Claude/Cline/etc. pedir decompilaciones y renombrar automáticamente. Hay alternativas (p. ej. `suidpit/ghidra-mcp`).
- La idea de «iterar con humano en el bucle» es lo que diferencia el flujo moderno del simple «pega el código en un chat». Es también lo que hará la demo (Acto A).

Slide 9 — Divisor: Manual vs. AI-Augmented Workflow

🎤 **QUÉ DECIR**

«Con las herramientas situadas, comparemos los dos flujos de trabajo.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Transición.

Slide 10 — Two lanes, same destination

🎤 **QUÉ DECIR**

«A la izquierda, el flujo **manual**: cargar el binario, mapear funciones a mano, leer el pseudocódigo línea a línea, deducir la intención, reproducir el comportamiento en sandbox y finalmente localizar el fallo y diseñar la mitigación. Horas o días, y la profundidad depende de la pericia del experto. A la derecha, el flujo **aumentado**: el agente decompila y resume funciones en bloque, el LLM explica y renombra, el analista dirige con preguntas, la IA redacta IOCs, reglas YARA o un parche… y el analista **verifica contra el binario**. El triaje baja a minutos, pero el paso de verificación sigue siendo obligatorio.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

El mensaje de este par de carriles: la IA **no elimina** pasos críticos, los **reordena**. El esfuerzo humano se desplaza de **leer** a **verificar**. Útil para responder «¿entonces sobra el analista?»: no, cambia su rol hacia validación y criterio.

Slide 11 — Comparison table

🎤 **QUÉ DECIR**

«Resumido en una tabla: la IA gana claramente en **velocidad de triaje**, en **comprensión para no-expertos** y en **cobertura** del código. El flujo manual gana en una cosa decisiva: la **garantía de corrección**, porque está verificado por el analista. Y el gran riesgo del carril de IA son las **alucinaciones**: errores dichos con total confianza. En una frase: la IA desplaza el esfuerzo de leer a verificar.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Las filas están pensadas para que cada una tenga un «ganador» distinto → evita vender la IA como superior en todo (eso es lo que el profesor premia como calidad técnica honesta). «Needs verification» y «Hallucinations» conectan directamente con la sección de evidencia.

Slide 12 — Divisor: Evidence — Speed, Accuracy & Hallucinations

🎤 **QUÉ DECIR**

«¿Y qué dicen los datos? Vamos a la evidencia.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Transición.

Slide 13 — Speed & comprehension

🎤 **QUÉ DECIR**

«Sobre velocidad y comprensión, un estudio reciente con 48 participantes haciendo ingeniería inversa (NDSS 2026) encontró que la comprensión de los **novatos** mejoró alrededor de un **98%**, y que las funciones que implementaban **algoritmos conocidos** se triaban hasta **2,4 veces más rápido**. Pero —y es importante— los **expertos** ganaron poco, y algunas respuestas incluían **alucinaciones dañinas**. O sea: la mayor ganancia es **bajar la barrera de entrada** y acelerar el **primer vistazo**, no reemplazar el juicio experto.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Estudio: «Decompiling the Synergy: An Empirical Study of Human–LLM Teaming in Software Reverse Engineering» (Eurecom, NDSS 2026) — ficha. Cifras: comprensión de novatos + 98%; hasta 2,4× más rápido en funciones con algoritmos conocidos; experts con poca mejora y presencia de hallucinations. Es el estudio con **sujetos humanos** más riguroso de los que manejamos → cítalo como el dato fuerte. Matiz honesto: los números dependen de la muestra y tareas concretas.

Slide 14 — Remediation & patching

🎤 **QUÉ DECIR**

«En **remediación y parcheo** la foto es más dura. El benchmark **PatchEval**, con 1.000 CVEs reales, encontró que el mejor modelo o agente solo arreglaba alrededor del **23%** de las vulnerabilidades. Eso sí, los modelos son **complementarios**: entre varios, se cubrían bastantes más. Y estudios de arquitecturas de parcheo comparan flujos fijos frente a agentes, intercambiando **precisión contra tiempo y coste en tokens**. La conclusión: la IA **propone** el parche; el humano y la batería de tests **disponen**.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

- **PatchEval** (arXiv 2511.11019, nov 2025) — enlace. 1.000 CVEs (2015–2025), 65 CWE; mejor resultado **23,0%** (53/230 en el subconjunto con sandbox); fuerte **complementariedad** (98 bugs resueltos por al menos un modelo). No mide tiempo de reloj.
- **Estudio de arquitecturas de parcheo** (arXiv 2603.01257, 2026): compara workflow fijo / un agente / multi-agente / agentes de código generales; analiza correctitud, **tiempo de ejecución** y **tokens**. Hallazgo útil: los workflows fijos son eficientes pero frágiles; multi-agente = más overhead; los agentes de código generales rinden mejor en conjunto.
- Si te preguntan por «tiempos de generar parches»: no hay un estándar de wall-clock citable con confianza; por eso la **demo** aporta nuestra propia medición cronometrada.

Slide 15 — The hallucination problem

🎤 QUÉ DECIR

«El gran problema son las **alucinaciones**. Tres formas típicas: **lógica inventada** —explicaciones plausibles pero falsas—; **renombrados engañosos** —una función llamada `decrypt_payload` que no descifra nada—; y **baja recompilabilidad**: el trabajo DecLLM subió el código decompilado que **vuelve a compilar** del 9% al 25%, es decir, la mayoría **ni siquiera reconstruye**. ¿La mitigación? **Verificar contra la verdad del binario**: recompilar, comparar comportamiento, ejecución simbólica, tests. Y esta es exactamente la razón por la que la IA sigue siendo poco fiable para **clasificar malware** automáticamente.»

📚 PROFUNDIZACIÓN Y REFERENCIAS

- **DecLLM** (ISSTA 2025) — PDF: recompilabilidad **9% → 25%** con contexto completo; nombra fallos como **attention degradation**, **shortcuts** y **hallucination**.
- **D-LiFT / D-Score** (arXiv 2506.10125) — enlace: los pipelines previos introducen errores sintácticos/semánticos nuevos; proponen **puntuar** precisión y legibilidad usando el compilador (sintaxis) y ejecución simbólica (semántica).
- Conexión con clasificación de malware: si el modelo alucina sobre lo que hace el código, **etiquetarlo** (familia/benigno-malicioso) hereda ese error. Por eso la verificación contra el binario es innegociable. Este es el punto que más conecta con el temario de la asignatura.

Slide 16 — Divisor: Live Demo

🎤 QUÉ DECIR

«Y ahora lo veamos en acción.»

📚 PROFUNDIZACIÓN Y REFERENCIAS

Transición a la demo.

Slide 17 — Demo: before / after

🎤 QUÉ DECIR

«Para dar perspectiva, el "antes": en 2017, frenar **WannaCry** consistió en que un analista leyera el código del malware **en directo**, durante un brote mundial, para encontrar el **dominio kill-switch**. Trabajo manual, bajo presión, horas. El "después" es nuestra demo, con **cronómetro en pantalla**, en dos actos: en el **Acto A** usamos Ghidra con un LLM vía GhidraMCP para explicar una muestra documentada en una **VM aislada** y sacar un resumen de comportamiento, IOCs y una regla YARA más la mitigación. En el **Acto B** cogemos un binario vulnerable **hecho por nosotros**: el LLM detecta el fallo en el pseudocódigo, **propone el parche**, recompilamos y el crash desaparece.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

- **WannaCry / kill-switch** (12-mayo-2017): Marcus Hutchins encontró, leyendo el malware, una comprobación de un dominio; al registrarlo, se frenó la propagación (Wikipedia: WannaCry). Úsalo como gancho del «lo que tardaba una persona». Matiz honesto: fue en parte **accidental** y hubo varios investigadores (Darien Huss); no des por hecho un cronómetro exacto de esa historia.
- Ambos actos van **cronometrados** para que el «antes/después» sea tangible. Detalle completo en `demo/demo-plan.md`.
- Enfoque **defensivo**: remediación = IOCs/YARA/mitigación y parcheo de binario propio. Nada de distribuir malware.

Slide 18 — Play recorded demo

🎤 **QUÉ DECIR**

«Vamos con el vídeo.» (Reproducir. Al terminar, una frase: «Fijaos en el cronómetro: el triaje y el parcheo que a mano llevan mucho más, aquí se hacen en minutos **con verificación**.»)

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Cumple el ítem «Demo (real/vídeo)» del profesor. Ten el vídeo **embebido o en local** para no depender de internet. Si algo falla, ten capturas de respaldo.

Slide 19 — Divisor: Takeaways

🎤 **QUÉ DECIR**

«Para cerrar, las ideas que os queréis llevar.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Transición.

Slide 20 — Takeaways

🎤 **QUÉ DECIR**

«Tres mensajes. Uno: los LLM **aceleran** la ingeniería inversa y **redactan** borradores de remediación, sobre todo en **triaje** y para **no-expertos**. Dos: **no** sustituyen al analista —el éxito parcheando es modesto, en torno al 23%, y las alucinaciones son reales. Tres: lo que hace que funcione es **mantener un humano en el bucle** y **verificar cada salida contra el binario**. Y esa misma razón explica por qué la IA todavía **falla al clasificar malware** de forma fiable.»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**

Cierre que amarra el hilo conductor (acelerador + verificación). Deja la última frase conectando con la clasificación de malware porque es el punto de la asignatura y da pie a preguntas.

Slide 21 — Credits & contributions

🎤 **QUÉ DECIR**

«Estos son los créditos y el reparto de trabajo del grupo.» (Lee brevemente quién hizo qué.)

📚 **PROFUNDIZACIÓN Y REFERENCIAS**
El profesor **exige** créditos y contribución por miembro (puede haber nota distinta por persona). Rellenad la tabla con los nombres y ajustad el reparto a lo que realmente hizo cada uno. Mencionad herramientas (Typst/Touying) y que las fuentes están en este guion.

Slide 22 — Thank you / Q&A

🎤 **QUÉ DECIR**
«Gracias. ¿Preguntas?»

📚 **PROFUNDIZACIÓN Y REFERENCIAS**
Preguntas probables y respuesta corta:

- **«¿No reemplazará esto a los analistas?»** → No a corto plazo; cambia el rol hacia verificación/criterio; el éxito autónomo es bajo (23% parcheo).
- **«¿No es peligroso dar malware a un LLM?»** → Trabajamos en VM aislada, con muestras documentadas y salida defensiva; nunca distribuimos binarios.
- **«¿Y las alucinaciones en producción?»** → Por eso se verifica contra el binario (recompilar, tests, ejecución simbólica).

Bibliografía y dónde ampliar información

Fuentes **verificadas** (arXiv/NDSS/repos reales) para rellenar y defender las diapositivas. Cada entrada indica a qué **slide** da soporte. Si solo vas a leer 2–3, empieza por los **SoK/surveys** (apartado E): dan el panorama y enlazan al resto.

A) Ingeniería inversa / decompilación asistida por LLM → slides 5, 8, 13, 15

- **Decompiling the Synergy: Human–LLM Teaming in RE** (NDSS 2026, Eurecom) — ficha. Estudio con 48 personas; novatos + 98% comprensión, hasta 2,4× en algoritmos conocidos, poca mejora en expertos, hallucinations. **El dato «fuerte» de velocidad/comprensión.** [Slide 13]
- **LLM4Decompile: Decompiling Binary Code with LLMs** (arXiv 2403.05286, EMNLP 2024) — abs · código. Modelos abiertos (1.3B–33B) para decompilar; benchmark **Decompile-Eval** (recompilabilidad/re-ejecutabilidad en vez de solo tokens); la variante **-Ref** refina la salida de Ghidra. [Slides 5, 8]
- **DeGPT: Optimizing Decompiler Output with LLM** (NDSS 2024) — paper · código. Mejora la **legibilidad** de la salida de Ghidra (−24,4% carga cognitiva; 62,9% de comentarios con semántica útil); usa 3 roles LLM + filtro MSSC para no cambiar la semántica. **Ejemplo concreto de «explicar/renombrar».** [Slides 8, 13]
- **DecLLM** (ISSTA 2025) — PDF. Recompilabilidad 9%⟶25% con contexto completo; nombra fallos: attention degradation, shortcuts, hallucination. [Slide 15]

B) Remediación / parcheo automático (APR) con LLM → slide 14

- **PatchEval** (arXiv 2511.11019, 2025) — abs. 1.000 CVEs reales; mejor modelo 23%; fuerte complementariedad entre modelos; sandbox para comprobar que no hay regresiones. [Slide 14]
- **A Systematic Study of LLM-Based Architectures for Automated Patching** (arXiv 2603.01257, 2026) — abs. Compara workflow fijo / 1 agente / multi-agente / agentes de código; mide correctitud, **tiempo** y **tokens**. Útil si te preguntan por coste/tiempo. [Slide 14]
- **A Survey of LLM-based Automated Program Repair** (arXiv 2506.23749, 2025) — abs. Clasifica 63 sistemas en 4 paradigmas (fine-tuning, prompting, procedural, agentic). **Buen mapa del área de parcheo.** [Slide 14]
- **LLM-based APR — Systematic Literature Review** (arXiv 2405.01466) — abs. 189 trabajos (2020–2025), 20 tipos de bug incluyendo vulnerabilidades de seguridad. [Slide 14]
- **SEC-bench** (benchmark de seguridad; vía survey 2510.09721) — tasas reportadas 18% (PoC) y 34% (patching). Cítalo con cautela (dato secundario) como refuerzo de «el parcheo autónomo es difícil». [Slide 14]

C) Alucinaciones, verificación y límites → slide 15

- **D-LiFT / D-Score** (arXiv 2506.10125) — abs. Los pipelines introducen errores sintácticos/semánticos nuevos; proponen **puntuar** precisión (compilador) y semántica (ejecución simbólica). **La idea «verifica contra el binario».** [Slide 15]
- Conexión con **clasificación de malware**: si el modelo alucina qué hace el código, la etiqueta (familia/benigno-malicioso) hereda el error → por eso la verificación es innegociable. Argumento de cierre. [Slides 15, 20]

D) Herramientas y puentes (Ghidra, MCP) → slides 7, 8 y demo

- **Ghidra** (NSA, open-source) — ghidra-sre.org · GitHub. Decompilador estándar gratuito. [Slide 7]
- **GhidraMCP** (LaurieWired) — GitHub. Puente Ghidra↔LLM vía MCP: listar/decompilar funciones, renombrar, cross-refs. [Slide 8, demo]
- **Model Context Protocol (MCP)** — modelcontextprotocol.io. Estándar para dar herramientas/datos a un LLM. [Slide 8]
- Alternativa: **suidpit/ghidra-mcp** — GitHub. Otro plugin MCP para Ghidra. [Slide 8]

E) Panorama general (SoK / surveys) — empieza por aquí

- **SoK: Potentials and Challenges of LLMs for Reverse Engineering** (arXiv 2509.21821, 2025) — abs. Revisa 48 estudios, taxonomía de 6 dimensiones, riesgos **dual-use**. **El mejor punto de partida.** [Transversal]
- **LLM for Software Security: Code/Malware Analysis & RE** (Jelodar et al., arXiv 2504.07137, 2025) — abs. Survey centrado en malware; datasets y modelos. [Transversal]

F) Contexto de malware y casos → slides 4, 17

- **AV-TEST — Malware Statistics** — enlace. Cifras de volumen (cientos de miles de muestras nuevas/día). [Slide 4]
- **WannaCry ransomware attack** — Wikipedia. Caso del kill-switch (12-may-2017, M. Hutchins). Matiz: hallazgo en parte accidental, varios investigadores. [Slide 17]
- **MITRE ATT&CK** — attack.mitre.org. Marco de técnicas para describir comportamiento del malware (útil para IOCs/mitigación de la demo). [Demo]
- **CWE** (Common Weakness Enumeration) — cwe.mitre.org. Para nombrar el fallo del binario de la demo (CWE-120/CWE-787, buffer overflow). [Demo]

G) Dónde seguir buscando (buscadores y venues)

- **arXiv cs.CR** (seguridad) — arxiv.org/list/cs.CR/recent · **cs.SE** (software) — arxiv.org/list/cs.SE/recent.

- **Google Scholar** — scholar.google.com (busca: «LLM decompilation», «LLM vulnerability repair», «LLM malware analysis»).
- **Papers with Code** — paperswithcode.com (benchmarks y código asociado).
- **Venues top** donde publican estos trabajos: **NDSS**, **USENIX Security**, **IEEE S&P (Oakland)**, **ACM CCS** (seguridad); **ISSTA**, **ICSE**, **FSE**, **ASE** (ingeniería de software/APR).
- **MalwareBazaar** (abuse.ch) — bazaar.abuse.ch. Muestras para investigación (usar con cuidado; ver `demo/safety.md`).

Nota: algunas cifras de benchmarks provienen de los abstracts; si vas a ponerlas en una diapositiva como dato exacto, ábrelas y confírmalas en el PDF original (sobre todo PatchEval, DecLLM y el estudio NDSS 2026).